

2026-07-19 Weekly Update: Signal Fusion Methods for the Multi-Hop Retrieval Gate

1. Background

A multi-hop question answering (QA) system decomposes a question into a sequence of sub-questions (hops), retrieves evidence and produces an intermediate answer for each hop in turn, and finally generates an answer from all intermediate results. A K-hop question requires K rounds of retrieval: at each hop, the current sub-question's embedding is used to run cosine-similarity retrieval over a pool of candidate passages, one candidate is selected as that hop's evidence, appended to the running reasoning prefix, and the process continues to the next hop.

Because the highest-cosine-similarity candidate is not always the correct evidence, the system includes a lightweight classifier (referred to below as the "gate"): after a candidate is selected and appended to the prefix, the LLM (Llama-3.1-8B-Instruct) performs a forward pass over the text "prefix + this hop's sub-question + candidate evidence," and the last-token hidden state at a chosen layer is used to train a classifier that predicts whether this hop's evidence selection was wrong. The gate's output can be used in two ways: (a) as a trigger signal — if it exceeds a threshold, retrieval is redone; (b) as a continuous score, linearly mixed with the cosine similarity to directly rerank candidates. The production pipeline

([retrieval/run_retrieval_exp_wavefront.py](#)) uses (b), with the formula:

$$\text{final_score} = \text{emb_score} - \text{lambda} * \text{gate_score}$$

where **emb_score** is the cosine similarity between the candidate and the sub-question, **gate_score** is the gate's predicted probability that this candidate's evidence selection is wrong, and **lambda** is a fixed weight (default 0.25).

This week's work addresses one question: two hidden-state signals with **different properties** can be extracted from the same LLM forward pass to train a gate. Both signals have already been validated independently, and this week tested several ways of combining them.

2. Two Independently Validated Signals

2.1 Local Change Signal Delta (gate v2)

For hop j , take the prefix hidden state before evidence selection, h_{j-1} , and the prefix hidden state after evidence selection, h_j , and define $\text{Delta}_j = h_j - h_{j-1}$ — how much the model's internal state changed as a result of this step's evidence choice. Training data comes from gold/counterfactual trace pairs sampled per (K, hop) bucket

([hidden_states/pilot_multilayer](#)); one **PCA(64) + Logistic Regression** model is trained per pooled hop position j ($j=0..3$, representing semantically equivalent hop positions across different K values). Layer selection was determined empirically: layer 15 for $j=0,1$ and layer 23 for $j=2,3$.

End-to-end evaluation (retrieval + generation) on the full musique dev set (2417 examples, --beam-width 1), across four metrics: hop-level recall@1, chain (fraction of questions where every hop's evidence is retrieved correctly), answer EM, and answer F1:

Method	recall@1	chain	EM	F1
baseline (no gate, cosine ranking only)	0.6513	0.4233	0.4079	0.5012
gate v2 (Delta, continuous weighted mixing)	0.6627	0.4357	0.4100	0.5045

2.2 Cumulative State Signal h_{after} (probe)

Unlike Delta, a second signal uses only the absolute hidden state after evidence selection, h_{after} (no differencing) — representing "the state of the entire reasoning prefix up to and including this hop." Training data additionally extracts the hidden state at **every** intermediate prefix position of a trace (not just the last hop), fixed at layer 31, and trains a PCA(64) + Logistic Regression model (error_propagation_probe/probe_artifacts).

Evaluated the same way on the full dev set, using $\text{emb_score} - 0.25 * \text{probe_score}$:

Method	chain	EM	F1
probe (h_{after} , weighted mixing)	0.4659	0.393	0.4816

Retrieval-side metrics beat baseline, but answer-side EM/F1 did not, indicating this signal alone provides limited benefit to the final answer.

Delta is a **local** signal (how much this specific step changed); h_{after} is a **global** signal (whether the overall state up to this point looks normal). An overlap analysis of the two signals' errors (aligned by example id, 2417 examples) shows that roughly 13-14% of examples are solved by only one of the two signals — a non-redundant, complementary coverage. Under a hypothetical perfect-selection oracle, EM could reach 47.3% (vs. roughly 39-41% for either signal alone), motivating the combination experiments below.

3. Combination Method 1: Pre-Retrieval Context-Conditioned Weighting (Not Adopted)

The first combination approach trained a small network to predict a [0,1] mixing coefficient from context available **before retrieval happens** (the main question embedding, the pre-retrieval prefix hidden state, the current sub-question embedding), determining how much to trust cosine similarity versus the gate score for that hop overall. The key structural limitation: the predicted mixing coefficient is **hop-level** — every candidate within the same hop's pool shares the same coefficient, so the model cannot condition on which specific candidate is being scored; it can only produce one blanket judgment for the entire hop.

Evaluated offline on dev using the candidate pool's gold rank (recall@1/recall@3/mrr, no generation involved):

	recall@1	recall@3	mrr
cosine similarity only	0.8067	0.9396	0.8778
fixed lambda=0.25 (production default)	0.8601	0.9635	0.9145
learned hop-level weight	0.8042	0.9318	0.8736

The learned weight did not beat the fixed-weight baseline, and training exhibited the coefficient converging to a boundary constant. Because the mixing coefficient is shared across the whole hop, the training objective degenerates into a single blanket judgment ("which side should be trusted overall"), lacking candidate-level differentiating signal, which drives the coefficient toward an extreme value rather than an informative intermediate one. Based on this, the "pre-retrieval context, hop-shared weight" design was judged structurally limited and not adopted.

4. Combination Method 2: Post-Hoc Fusion with Fixed / Heuristic Weights

Post-hoc fusion means gate v2 and the probe are each trained independently, and only their output scores are combined at inference time — no joint training is involved.

4.1 Three Voting Schemes (`combined_gate_rerank`)

Both signals are scored within a single batched forward pass (union of required layers: gate v2's {15, 23} plus the probe's {31}, sharing one Llama forward computation). Combination schemes:

- **sum**: $\text{final} = \text{emb_score} - \text{lambda_1} * \text{gate_v2_score} - \text{lambda_2} * \text{probe_score}$, $\text{lambda_1} = \text{lambda_2} = 0.25$
- **veto**: any candidate flagged as abnormal by either model (probability > 0.5) is excluded; the remaining candidates are ranked by cosine similarity
- **rfr**: gate v2 and the probe each independently produce a complete rerank first, and the two resulting rank **positions** (not raw scores) are fused via Reciprocal Rank Fusion ($1/(k+\text{rank_1}) + 1/(k+\text{rank_2})$, $k=60$), without comparing score magnitudes

Full musique dev end-to-end results:

Method	recall@1	chain	EM	F1
sum	0.6869	0.4894	0.4141	0.5089
rfr	0.6705	0.4564	0.3922	0.4822
veto	no improvement observed			

sum is the only scheme that simultaneously beats both gate v2 and the probe used alone. To verify whether sum's gain reflects genuine complementarity (rather than simply leaning on the stronger signal), the retention rate of each signal's uniquely-correct examples and the recovery rate among examples where both signals are wrong were measured: the retention rates for gate v2-only-correct and probe-only-correct examples were 47.2% and 78.1% respectively, and among the 1186 examples where both signals were wrong, sum recovered 4.9% (about 58

examples) — cases that neither signal alone could solve, confirming a genuine complementary effect.

4.2 Finer-Grained Heuristic Weighting

Building on sum, three further weighting refinements were tested, all evaluated on the same real retrieval candidate-pool dataset (built by `adaptive_lambda_rerank`, containing each candidate's cosine score, gate v2 score, and probe score), with candidate-pool gold-rank recall@1 as the metric:

- **Confidence-weighted**: dynamically weight by the within-pool standard deviation of each signal's scores (a larger spread indicates a more decisive judgment this time)
- **Coarse-then-fine**: first shortlist top-k candidates using the probe score, then rerank the shortlist using the gate v2 score
- **Grid search**: independently sweep (lambda_gate_v2, lambda_probe) combinations

Method	recall@1
sum (fixed 0.25/0.25)	0.8660
confidence-weighted	0.8646
coarse-then-fine (k=3)	0.8643
grid-search optimum (0.25, 0.10)	0.8690

The grid-search configuration slightly beats sum on overall recall@1, but a complementarity analysis of this configuration shows the probe's unique-win retention rate drops to 50.6% (vs. sum's 72.1%), and the recovery rate among examples where both signals are wrong drops to 1.5% (vs. sum's 5.5%) — this configuration trades away complementarity for a marginal gain in overall accuracy, at odds with the goal of preserving each signal's distinct contribution.

A three-way convex combination was also tested (cosine, probe, and gate_v2 weights summing to 1, each z-scored before combining): the best overall recall@1 configuration was (0.6, 0.2, 0.2) -> 0.8679; an equal-thirds configuration (0.33, 0.33, 0.33) -> 0.8463 (lower overall accuracy), but with a both-wrong recovery rate of 16.0% (substantially higher than any other configuration). This series of experiments reveals a consistent pattern: **the more a weight configuration is optimized for overall accuracy, the lower its complementary recovery rate; the more balanced the weighting, the higher the recovery rate but the lower the overall number** — every post-hoc fusion method tested so far sits on this same trade-off curve, without improving both simultaneously.

5. Feature-Level Fusion (gate v3)

All methods above are post-hoc fusion: gate v2 and the probe are trained independently, and only their **output scores** are combined at inference. gate v3 instead performs feature-level fusion: rather than training two independent models, the same hop's `h_after` (the probe's raw feature) and `Delta_j` (gate v2's raw feature) are each independently projected with PCA (their covariance structures differ, so they do not share one PCA), the two reduced feature sets are concatenated, and a **single** Logistic Regression is trained:

$$p_j = \text{sigmoid}(w_h^T \text{PCA}(h_{\text{after}}) + w_{\text{delta}}^T \text{PCA}(\Delta_j) + b)$$

Layer selection reuses gate v2's already-validated per-position layers (layer 15 for $j=0,1$; layer 23 for $j=2,3$); training data reuses the already-extracted `hidden_states/pilot_multilayer`, requiring no new hidden-state extraction.

5.1 Classification-Metric Evaluation

Under the same evaluation protocol as gate v2 (fixed gold/counterfactual trace data, binary classification of "was this hop's evidence selection wrong"):

	AUC	Overall F1	Overall TPR
gate v2 (Delta only)	0.9221	0.7785	0.8456
gate v3 (h_{after} (+) Delta)	0.9357	0.793	0.8838

Broken down by hop position, $j=2$ (gate v2's weakest position) shows the largest F1 gain (0.7535 $\rightarrow$ 0.7919).

5.2 End-to-End Evaluation

Full musique dev, using the same formula structure as gate v2, `final_score = emb_score - lambda * gate_v3_score` ($\lambda=0.25$), with the single Delta score simply replaced by the score from the concatenated feature:

Method	recall@1	chain	EM	F1
gate v2 (Delta only)	0.6627	0.4357	0.4100	0.5045
gate v3 (h_{after} (+) Delta)	0.6950	0.5010	0.4224	0.5146

All four metrics improve over gate v2, with chain showing the largest gain (+6.5 percentage points), consistent with the classification-metric result where $j=2$ improved the most: the longer the reasoning chain, the more a single weak hop is amplified, so improving the weakest hop has the largest effect on overall chain accuracy.

6. Learned Per-Candidate Gating (context gate)

The structural reason Section 3's method failed was that "the weight is shared per hop and cannot see the specific candidate." This section re-designs the same idea at the **candidate level**: for each candidate evidence passage, a mixing coefficient is computed from **that candidate's own** post-selection `h_after` (rather than pre-retrieval context):

```
alpha_i = sigmoid(w^T PCA(h_after)_i + b)           # computed separately
per candidate
anomaly_i = alpha_i * gate_v2_score_i + (1 - alpha_i) * probe_score_i
```

```
final_i      = emb_score_i - 0.5 * anomaly_i      # 0.5 = sum's fixed
lambda_1+lambda_2 budget
```

Because different candidates within the same hop append different evidence text, each has its own distinct `h_after`, so alpha can take different values across candidates within the same pool — this is the key structural difference from Section 3's method, giving training a genuine candidate-level differentiating signal. The model is a single linear layer plus sigmoid (65 parameters), trained with a listwise cross-entropy loss on candidate-pool gold rank. The required per-candidate `h_after` (PCA-projected to 64 dimensions, reusing the probe's own PCA and layer 31) was obtained by extending the existing candidate-pool dataset; no new Llama forward passes beyond what was already computed were required.

6.1 Offline Candidate-Pool Ranking Evaluation

Using the same candidate-pool recall@1 metric as Section 4:

Method	recall@1	Recovery rate when both signals are wrong
sum (fixed 0.25/0.25)	0.8660	5.5%
context gate (learned per-candidate weight)	0.8776	16.4%

The learned alpha has mean 0.5593 and standard deviation 0.4636, spanning the full [0,1] range with no sign of collapsing to a constant. This is the only combination method tested this week that simultaneously improves both overall recall@1 and the complementary recovery rate, without the trade-off seen in every other post-hoc fusion method.

6.2 End-to-End Evaluation

Full musique dev:

Method	recall@1	chain	EM	F1
gate v3	0.6950	0.5010	0.4224	0.5146
context gate	0.7018	0.5184	0.4208	0.5147

Retrieval-side metrics (recall@1, chain) exceed gate v3, with chain improving by 1.74 percentage points, the largest gain of any method this week. Answer-side metrics (EM, F1) are essentially on par with gate v3, with EM 0.16 points lower (within noise). The retrieval-side gain did not fully translate into an answer-side gain, plausibly related to a capacity limit in the final reading-comprehension module itself (EM uses strict string matching).

7. Summary of Results

End-to-end evaluation (full musique dev, `--beam-width 1`, consistent metric definitions across all methods):

Method	recall@1	chain	EM	F1
baseline	0.6513	0.4233	0.4079	0.5012
gate v2 (Delta)	0.6627	0.4357	0.4100	0.5045
probe (h_after)	—	0.4659	0.393	0.4816
post-hoc sum (gate v2 + probe, fixed weights)	0.6869	0.4894	0.4141	0.5089
post-hoc rrf	0.6705	0.4564	0.3922	0.4822
gate v3 (feature-level fusion)	0.6950	0.5010	0.4224	0.5146
context gate (learned per-candidate weight)	0.7018	0.5184	0.4208	0.5147

gate v3 and context gate perform similarly end-to-end: gate v3 is marginally better on answer-side metrics, context gate is better on retrieval-side metrics. Given comparable performance, gate v3 has a simpler structure (a single Logistic Regression reusing an already-validated layer selection, trained in one sklearn fit) and requires no additional neural gating module; context gate requires maintaining three components — gate v2, the probe, and a separately-trained gating network — a more complex pipeline overall. Weighing both factors, gate v3 is adopted as the current default, with context gate retained as a validated alternative.